

RTTAC-An Access Control Model Based on Blockchain and Role-Trusted Transfer

Feng Wang

Henan university of technology, Distributed Computing &
BlockChain Lab, Zhengzhou, China
wfmail@sina.com

Yang Liu*

Henan university of technology, Distributed Computing &
BlockChain Lab, Zhengzhou, China
*liu_yang@haut.edu.cn

Yang Lu

Henan university of technology, Distributed Computing &
BlockChain Lab, Zhengzhou, China
ly1999@stu.haut.edu.cn

Zhiruo Zhang

Henan university of technology, Distributed Computing &
BlockChain Lab, Zhengzhou, China
ZhangZhiruo_1999@163.com

Abstract—Aiming to address the data security issues in data sharing, current approaches such as RBAC, ABAC, and blockchain-based data sharing platforms suffer from problems like role redundancy, role abuse, complex authorization, and data user privacy leakage. In response, a role-based trusted transfer access control model based on trust and attributes is proposed. This model is built upon Attribute-Based Access Control (ABAC) while integrating Role-Based Access Control (RBAC). Users obtain roles based on their attributes and trust levels, and roles determine access permissions based on trust level and defined policies, and then determine the transferability of roles, at the same time, the transfer of roles is limited by trust levels. Encryption techniques and off-chain storage systems are utilized to protect data privacy and integrity, thereby alleviating the storage pressure on the blockchain. Finally, the performance and effectiveness of the proposed model were evaluated through experimental analysis, demonstrating its capability to achieve trustworthy role transfer and enhance the security and privacy of on-chain sharing schemes.

Keywords— *Blockchain; Trust level; Role transfer; Role-based access control; Attribute-based access control*

I. INTRODUCTION

With the development of information technology, data sharing among organizations faces challenges. Different definitions of data and data silos increase communication costs, and data security is often threatened. The development of blockchain technology has led to its widespread application in information security fields such as data sharing[1]. However, challenges exist in this area, including issues related to data transparency, incomplete privacy protection, and the potential for malicious activities that could lead to privacy breaches and information linkage problems[2].

In recent years, the rapid development of blockchain technology, with its characteristics such as immutable data management, decentralized distributed storage, smart contracts, and trusted traceability, has found wide applications in the field of information security, including identity authentication, access control, and data protection. This aligns well with the need for data sharing. However, migrating data sharing to the blockchain is not the ultimate solution, as on-chain data transparency poses privacy challenges that concern researchers.

Access control technology plays a crucial role in maintaining data security. It ensures data security and the smooth operation

of business systems by managing user permissions and restricting unauthorized access. In recent years, research on access control technologies such as Role-Based Access Control (RBAC) and Attribute-Based Access Control (ABAC) has made significant progress both domestically and internationally.

Traditional RBAC is limited by predefined authorization patterns, making it difficult to adapt to complex and dynamic access environments. As the system scales up and becomes more complex, it can lead to an explosion in the number of roles, resulting in what is known as a "role explosion." While traditional ABAC achieves fine-grained access control, it is complex and requires managing multiple attribute combination policies. Additionally, its dynamic nature makes permission management even more complicated.

This paper addresses the issues described above and the current research by designing an RBAC role transfer mechanism and proposing an access control model called RTTAC (Role Trusted Transfer Access Control) based on blockchain technology. The primary idea is:

- Integrating the RBAC and ABAC models. ABAC provides RBAC with dynamic and fine-grained access control while reducing the complexity of permission management in ABAC.
- Establishing a mechanism for assessing user trustworthiness. Integrating this mechanism into role attributes to achieve trusted role transfer.
- RTTAC is integrated with blockchain to ensure that operations such as user authorization and role transfer are automatically executed by smart contracts and recorded on the blockchain. In addition, by leveraging the IPFS of f-chain storage system, only data digest values are stored on the blockchain.

II. RELATED WORK

The RBAC (Role-Based Access Control) model[3][4] achieves the granting and revocation of user access permissions by assigning and revoking roles for users. The ABAC (Attribute-Based Access Control) model is to determine access not only based on user attributes but also by considering object attributes, environmental conditions, and whether the current values align with the access policies established by the object owner.

Jin et al. [5] proposed a role-centric attribute-based access control (RABAC) model that defines a Permission Filtering Policy (PFP). The PFP adjusts the set of permissions available to a user's roles based on whether user and object attributes comply with the policy. Rajpoot et al. [6] introduced a role-centric access control model that combines RBAC with enhanced attributes from ABAC. This model allows for the consideration of request context when making access control decisions, but it has limitations related to the potential misuse of context information. Xiao et al. [7] proposed an access control scheme that is based on blockchain and ABAC. This scheme utilizes smart contracts to implement the Policy Decision Point (PDP) as seen in traditional ABAC, and it employs a dual-chain structure comprising a data chain and a policy chain. However, managing authorization becomes challenging in complex environments. Jiang Jiahao et al. [8] proposed a hybrid access control scheme for data sharing based on attributes and capability tokens. This approach allows users to delegate capability tokens to each other, enhancing the flexibility of a single access control model. But it doesn't address the issue of arbitrary token delegation. Zhang Lejun et al. [9] proposed a trust evaluation mechanism integrated into access control, enabling secure data access and sharing through smart contract integration. But their approach doesn't offer fine-grained access control.

III. SYSTEM MODEL

Divided into Attribute Management(AMC), Policy Management(PMC), and Role Management(RMC).

AMC collects various attribute values submitted to the blockchain and integrates them into an attribute structure, which is then stored on the blockchain to provide attribute authority authentication.

PMC gathers access control policies uploaded to the blockchain by Data Owners (DO). It integrates the policies based on attribute information consolidated by AMC and stores them on the blockchain, serving as a prerequisite for access request judgments.

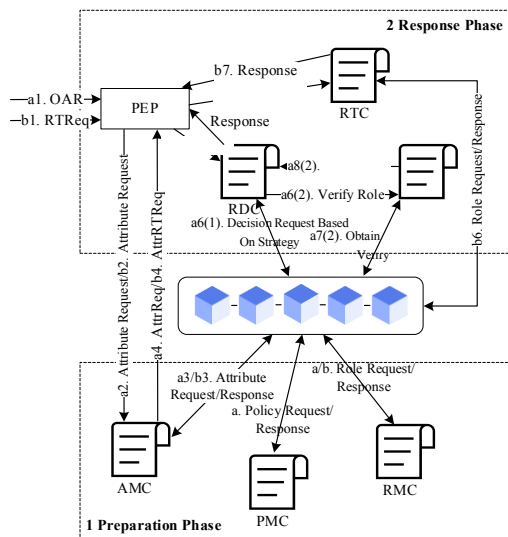
Role creation and management are handled by RMC. It constructs roles based on data attributes and corresponding policies, and then manages the collection of roles and permissions associations. The role attributes contained are provided to AMC for attribute management.

Divided into Data Access and Role Transfer.

a) PEP receives the original access request from the Data Requester (DR), known as the Original Access Request (OAR). It parses information from OAR, including subject, data, environment, operation, and role details. PEP requests relevant attribute information from AMC to construct an Attribute-Based Access Request (AttrReq) for OAR. Subsequently, AttrReq is sent to RDC.

b) RDC assesses whether DR possesses a role. If a role is held, it verifies the role based on information returned by RMC and assesses the access request according to policy information retrieved from PMC. If DR does not have a role, it triggers RAC to perform role assignment.

c) RAC validates the access request based on the request and role information returned by RMC. It also verifies the request against policies from PMC. Finally, role assignment is executed, and the results are relayed back to RDC, along with



updates to the trust values of relevant subjects associated with DR.

d) RDC returns the execution results to PEP.

2) *Role Transfer*: Role transfer is another form of action in the response phase. It doesn't involve inheritance or delegation. Roles serve as carriers of permissions, and their permissions are determined when the system is initialized. Role transfer means transferring the ownership of a role from the original Data Requester (DR) to another DR. The permissions carried by the role remain unchanged, only the ownership of the role changes. As shown in Figure 1b's process.

a) PEP receives a role transfer request from DR. Based on the subject attributes returned by AMC, it interprets the request as a transfer request for the corresponding DR attribute. Then, it sends the request to RTC.

b) RTC executes the role transfer operation based on the request. It verifies the reasonableness of the role transfer request by calling RMC to obtain relevant role information.

c) RTC returns the execution results to PEP.

IV. RTTAC DESIGN

A. Evaluation of User Trust Level for RTTAC

The RTTAC model introduces a trust assessment mechanism to restrict users. In this model, a user's trustworthiness is represented by a total trust value, which comprises several components: current trust value (*Current_TV(U)*), historical trust value (*History_TV(U)*), and reference trust value (*Refer_TV(U)*). These components together form the user's total trust value (*Total_TV(U)*).

1) User trust value calculation:

a) *Calculation of Current Trust Value*: Calculating a user's trust value requires considering various factors, and the relationships and weights between these factors are uncertain. The RTTAC model uses the Fuzzy Analytic Hierarchy Process (FAHP) to compute the user's current trust value [9, 10]. FAHP decomposes the DR's access behavior into multiple distinct characteristics and further breaks down each characteristic into several items of evidence. This refinement process transforms the fuzzy trust value assessment problem of DR's access behavior into a well-defined and intuitive problem of weighted evidence summation. The main calculation steps are as follows:

Step0: Establishing a Multi-Level Structure Model

Based on the factors that influence the trust value of DR, the user's trust value is categorized into three characteristics. Each of these characteristics is further divided into various pieces of evidence that provide a more intuitive reflection of DR's access behavior, as shown in Figure 2. To facilitate the calculation of DR's current trust value, all the illustrated evidence values need to be normalized into dimensionless values that increase monotonically along the positive direction within the [0, 1] interval.

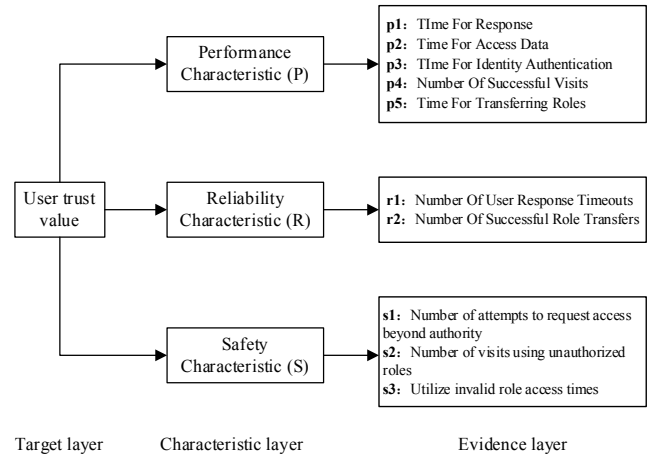


Figure 2. RTTAC framework

Step1: Calculate the Initial Judgment Matrix

Let the normalized evidence matrix be denoted as $E = [e_{ij}]_{mn}$, where $e_{ij} \in [0,1]$ represents the j evidence of the i characteristic. Here, m is the number of characteristics, and n is the number of pieces of evidence under that characteristic. Taking the performance characteristic P as an example, the evidence matrix for this characteristic is $E_P = [e_1, e_2, e_3, e_4, e_5]$. Using this matrix, the initial judgment matrix $EQP = [eq_{ij}]_{nn}$ is obtained by comparing the evidence values pairwise for the same characteristic, where eq_{ij} represents the importance of e_i and e_j regarding the performance characteristic. Here, $n = 5$, and the initial judgment matrix EQP is determined using equation (1).

$$eq_{ij} = \begin{cases} 0, & e_i < e_j \\ 0.5, & e_i = e_j \\ 1, & e_i > e_j \end{cases} \quad (1)$$

In this context, where $e_i < e_j$ indicates that evidence e_i is less important than evidence e_j , $e_i = e_j$ implies that evidence e_i and e_j are equally important, and $e_i > e_j$ signifies that evidence e_i is more important than evidence e_j .

Step2: Calculate the Weights of Each Piece of Evidence and Each Characteristic

For the initial judgment matrix $EQP = [eq_{ij}]_{nn}$, perform row summation and row transformation to obtain the fuzzy consistency matrix $QP = [q_{ij}]_{nn}$ using equation (2).

$$\begin{cases} q_i &= \sum_{k=1}^n eq_{ik} \\ q_{ij} &= \frac{q_i - q_j}{2n} + 0.5 \end{cases} \quad (2)$$

Using equation (3), calculate the weight vector for each piece of evidence under the performance characteristic based on the obtained fuzzy consistency matrix QP .

$$w_i = \frac{1}{n(n-1)/2} [\sum_{k=1}^n q_{ik} - 0.5] \quad (3)$$

For the performance characteristic P , the weight vector for the evidence is denoted as $W_P = [w_1, w_2, w_3, w_4, w_5]^T$.

Similarly, following the above steps, evidence weight vectors $\mathbf{W}_R$ and $\mathbf{W}_S$ can be obtained for the reliability characteristic R and security characteristic S , respectively. Likewise, you can calculate the weight vector for each characteristic's influence on the top-level user trust value, denoted as $\mathbf{W}_f = [w_{f1}, w_{f2}, w_{f3}]$.

Step3: Calculate the User's Current Trust Value

First, you need to calculate the evaluation matrix $\mathbf{F}$ for the characteristics using the normalized evidence matrix $\mathbf{E} = [e_{ij}]_{mn}$ and the weight matrix $\mathbf{W} = W[w_{ij}]_{mn}$. Then, based on the weight matrix and the evaluation matrix for the characteristics, you can calculate the direct trust values. Taking this hierarchical structure as an example, the weight matrix for the evidence $\mathbf{W}[w_{ij}]_{mn} = W[w_{ij}]_{3 \times 5} = (W_p, W_R, W_S)^T$. Calculate $\mathbf{E} \times \mathbf{W}^T$ to obtain the diagonal values of the resulting matrix, which is the evaluation matrix $\mathbf{F} = [f_1, f_2, f_3, \dots, f_m]$.

Finally, calculate the current trust value of the user according to equation (4).

$$\text{Current_TV}(U) = F \times W_f = \sum_{i=1}^m f_i \times w_{fi} \quad (4)$$

b) *Calculation of Historical Trust Value:* When DR first logs into the system for access control, the system sets the user's initial $\text{History_TV}(U)$ to 0. As DR's access frequency increases, the historical trust value of DR changes over time by the changes in access time and the final trust value:

$$\text{History_TV}(U) = \begin{cases} 0, & n = 0 \\ \sum_{i=1}^n (\text{Total_TV}_i(U) \times t_i) / \sum_{i=1}^n t_i, & n > 0 \end{cases} \quad (5)$$

In equation (5), " n " represents the number of accesses, and " t_i " records the access time. This parameter signifies the time interval from the start of DR's access until the end when they exit. When $n = 0$, DR's $\text{History_TV}(U)$ is set to 0, indicating that DR is making its first request for data, and there is no corresponding historical trust value.

c) *Calculation of Reference Trust Value:* When DR initially accesses data without any historical trust value, the reference trust value can be calculated to determine the user's trustworthiness based on their interactions with other organizations. Suppose the user has interacted with n organizations, and their total trust value with the organization i is TV_i , and the reliability weight of that organization is O_i . In that case, the user's reference trust value is calculated as follows:

$$\text{Refer_TV}(U) = \sum_{i=1}^n (\text{TV}_i \times O_i) \quad (6)$$

2) *Classification of user trust level evaluation*

Based on the current trust value and historical trust value calculated according to section 3.1, these two values serve as the foundation for determining DR's $\text{Total_TV}(U)$ comprehensively:

$$\text{Total_TV}(U) = x \times \text{Current_TV}(U) + y \times \text{History_TV}(U) + z \times \text{Refer_TV}(U) \quad (7)$$

In the equation, $x + y + z = 1$, with $x > y > z$, when DR is accessing the system for the first time, the user's $\text{History_TV}(U)$

is set to 0. When DR has not accessed other organizations, the user's $\text{Refer_TV}(U)$ is 0. However, when DR is not accessing the system for the first time, the values of x , y , and z dynamically change, and the extent of change depends on the operations requested by DR and the current trust value. If DR's requested operation carries a higher risk, and the current trust value is lower, then x tends to approach 1, while the weights of y and z tend to approach 0. This adjustment is implemented to prevent malicious DRs from artificially inflating their historical trust value to increase their total trust value and gain higher permissions.

To facilitate the verification of DR's identity information in subsequent access control experiments, trustworthiness levels are established based on DR's total trust value, as shown in Table 1.

TABLE I. CLASSIFICATION OF USER TRUST LEVELS

Trust Level	Trust Value Range	Accessible Data Types
0	(0, 0.25]	None
1	(0.25, 0.5]	PublicData
2	(0.5, 0.75]	WritableData
3	(0.75, 1]	InternalData

B. RTTAC Smart Contract Design

1) *Data Storage*

The InterPlanetary File System (IPFS) is a global, peer-to-peer, distributed versioned file system [11]. It utilizes the Merkle DAG (Directed Acyclic Graph) data structure to manage data, where leaf nodes store the hash values of multiple data blocks. By using the hash values of leaf nodes to construct the hash values of parent nodes iteratively, the root node's hash value is eventually obtained, representing the entire data structure.

The IPFS network is distributed. After a Data Owner (DO) uploads a file to the IPFS network, other nodes in the network can download and store it locally. This could potentially lead to data leakage. To prevent this, DO encrypts the data using a proxy re-encryption algorithm before uploading it to the IPFS network. Finally, the Content Identifier (CID) is submitted as a transaction to the blockchain.

Algorithm 1 DataStore

Input: Certificate, DOPubKey, Data

Output: CID or FALSE

```

1: ResStore = FALSE
2: If certIsAuthorized(Certificate) // verify digital certificate
3:   cipherText, err = encrypt(Data, DOPubKey)
4:   shell = shell.NewShell("localhost:5001")
5:   CID, err = shell.Add(Data)
6: If err = nil
7:   ctx.GetStub().PutState(H(CID), CID)
8:   return CID
9: return ResStore

```

2) *Preparation Phase*

Definition 1: Attribute ($Attr$) is a variable that specifies the data type and value domain. It is represented as $xAttr$ and $xAttrSet$, where x can be one of $\{Sub, Data, Envir, Act, Role\}$, representing attributes and attribute sets for subjects, data, environment, actions, and roles, respectively.

AMC collects and integrates subject attributes (*SubAttr*) and data attributes (*DataAttr*). Notably, the total trust value (*SubTotalTv*) is updated after each operation in the response phase, providing the latest total trust value for subsequent operations. AMC also defines methods such as *CreateSubAttr()*, *UpdateSubAttr()*, *GetSubAttr()*, *DelSubAttr()*, *CreateDataAttr()*, *UpdateDataAttr()*, *GetDataAttr()*, and *DelDataAttr()* to provide functionality for managing attributes.

The member attributes of *DataAttr* can be categorized into *PublicData*, *WritableData*, and *InternalData* data types. The definition and creation of *SubAttr* and *DataAttr* are executed by the DO. In *SubAttr*, *SubOther* describes additional information about the subject, such as the subject's location. In *DataAttr*, *DataOther* describes other attributes of the data, such as its data type. To ensure the uniqueness and security of subject attributes, *SubAttr* uses the hash value of the organization member's digital certificate as the unique ID for subject attributes. The value types of *EnvirAttr* and *ActAttr* attributes are determined by the specific requirements of the model.

PMC is responsible for storing and managing the Access Authorization Policies (AAP) defined by the DO, and handling policy management. In the AAP, *EnvirAttr* represents time constraints and is defined using a time interval denoted as *BeginEnd*. *TrustLevel* specifies the minimum allowed access trust level for DR (Data Requester), and *ActAllow* defines the operations that DRs meeting the access policy rules are allowed to perform. Within the AAP, *SubAttr*, and *DataAttr* specify specific subjects and data that can be accessed. DO can modify the unique IDs in *SubAttr* and *DataAttr* properties to relax restrictions, enabling access control between multiple subjects and data. PMC also defines methods such as *CreatePolicy()*, *UpdatePolicy()*, *GetPolicy()*, and *DelPolicy()* for policy management.

Definition 2: Role (*Role*) is a collection of permissions that defines the operational privileges a user possesses. In this paper, Role is represented as $Role = \{RoleAttr, DataAttr, Permission\}$, where Role includes basic attributes such as *RoleName*, *UserName* (the role's owner), *TrustLevel* (trust level of the role), *Sign* (digital signature for role assignment), and *TransferTime* (the number of times the role can be transferred). *DataAttr* represents a set of data attributes that the role can access, and *Permission* represents the operational permissions the role can execute.

Roles are created to assign them to users, and the relationships between roles, subjects, and data are illustrated in Figure 3, where a role can correspond to 1 or n data items.

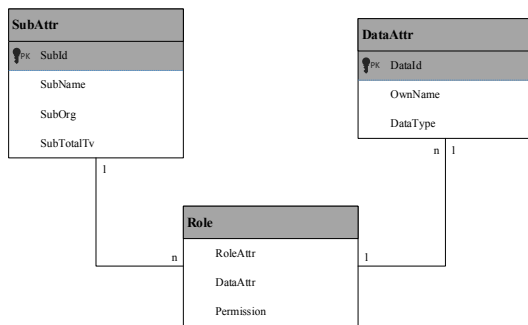


Figure 3. RTTAC framework

RMC creates roles based on the access policies (AAP). According to the definition, *RoleAttr* grants permissions for the role to execute Permission on *DataAttr*. In this context, Read, Write, and Delete permissions correspond to *RoleName Superuser*, *Miduser*, and *Lowuser*, respectively. *UserName* and *Sign* are initially empty, and *TransferTime* for these three roles, *Superuser*, *Miduser*, and *Lowuser*, are set to 1, 2, and 3, respectively. *TrustLevel* is set to the minimum trust level of the role owner. *DataAttr* represents the data types that the role can access. *DataId* can be specified to restrict access to specific data, or if it is left empty, *OwnName* and *DataType* define a series of accessible data. RMC provides the *isExistRole()* method to check if a role already exists.

3) Response Phase

Definition 3: Attribute Access Request (*AttrReq*). In this model, the attributes of the role (Role) reflect the permissions possessed by the visitor. *AttrReq* is represented as $AttrReq = \{SubAttr, Role, DataAttr, EnvirAttr, ActAttr\}$.

a) Data Access

To verify whether *AttrReq* complies with the policy requirements, if it does, assign a role to DR. Role assignment is done by invoking the *AssignRole()* method, which uses the DO's organization private key *OrgPriKey* to generate a role signature *Sign* and completes the assignment. Finally, recalculate DR's total trust value *NewTotalTv* and update its subject attributes.

Algorithm 3 RDC

Input: *AttrReq*

Output: *NewTotalTv*, *cipherTextRe*, *Result*

```

1: TotalTv = calculateTTv(SubAttr, calculateCTv(SubAttr))
2: TrustLevelDR = TLEvaluate(TotalTv)
3: certIsAuthorized(AttrReq.SubAttr) // verify digital certificate
4: AAPSet = GetPolicy(AttrReq, TrustLevelDR)
5: for AAP := range AAPSet
6:   If AttrReq matches AAP
7:     If IsMatchedRole(AttrReq.Role, OrgPubKey,
AttrReq.SubAttr, TrustLevelDR)
8:       Result = TRUE
9:   If Role = ""
10:    If RAC AssignRole Successful
11:      Result = TRUE
12:  When Result = TRUE
13:    cipherTextRe = ReEncrypt(RePriKey, cipherText)
14:    NewTotalTv = calculateTTv(SubAttr)
15:    UpdateSubAttr(SubAttr, NewTotalTv)
16: return NewTotalTv, cipherTextRe, Result
  
```

RDC performs access request judgment for DRs who possess roles. The judgment result, denoted as "Result," can be either "TRUE" or "FALSE." RDC receives the escaped *AttrReq* and then assesses whether the attributes in *AttrReq* conform to the role and policy information returned by RMC and PMC. When the attributes in *AttrReq* satisfy the attributes specified in the role and policy, the judgment result, "Result," is set to "TRUE." Subsequently, DO generates the re-encryption key *RePriKey*, and the organizational administrator conducts the re-encryption. Finally, based on the judgment result, DR's total trust value is recalculated and updated.

b) Role Transfer

Algorithm 4 RTC

Input: RTReq(DRa)

Output: NewTotalTvDRa, NewTotalTvDRb, RoleDRb or FALSE

```

1: certIsAuthorized(DRaSubId and DRbSubId) // verify digital
   certificate
2: TotalTv = calculateTTv(DRa,bSubId, calculateCTv(DRa,
   bSubId))
3: TrustLevelDR = TLEvaluate(TotalTv)
4: If IsMatchedRole(RoleDRa, OrgPubKey, DRaSubAttr,
   TrustLevelDRa)
5:   RoleDRb, err = checkDR(RoleDRa, DRbSubAttr,
   TrustLevelDRb, OrgPriKey)
6:   If err = ""
7:     RoleDRb.RoleAttr.TransferTime -- // Transfer Role
8:   NewTotalTvDRa,b = calculateTTv(DRa,bSubId)
9:   UpdateSubAttr(DRa,bSubId, NewTotalTvDRa,b)
10: return NewTotalTvDRa,b, RoleDRb

```

Superuser, *Miduser*, and *Lowuser* are three roles in which *Lowuser* has the lowest privileges with a transfer limit of 3 times. *Lowuser* can only access public data, making its permission set relatively limited. *Miduser*, on the other hand, can be transferred twice and has access to both public and writable data. *Superuser* has the highest level of permission, allowing access to internal data, but it can only be transferred once.

Assuming DRa is the one with the role and DRb is the recipient of the transfer, RTC invokes the IsMatchedRole() method to verify the legitimacy of DRa's role. If it is legitimate, RTC updates role-related attribute parameters, signs the new role, and finally sends the new role to DRb.

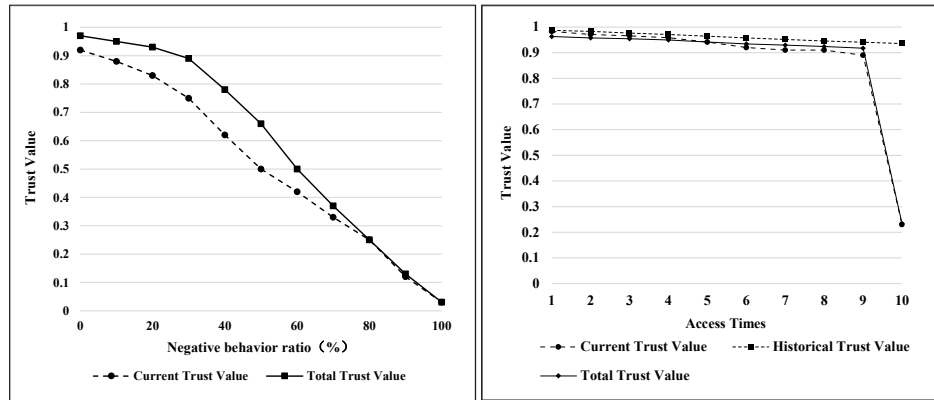


Figure 4. Change in Trust Values with Different Negative Behavior Ratios and Multiple Accesses

In a stable system operation, as users continuously perform various access operations, their trust values undergo corresponding changes. As shown in Figure 4, for the first 9 access requests, the trust value changes only slightly. However, on the 10th access, the user suddenly performs a risky operation that affects the security feature, causing a sharp decrease in the current trust value. Nonetheless, the historical trust value remains relatively stable, and the reference trust value remains fixed. This aligns with the conditions where a user executes a risky operation with a very low current trust value. As a result, the user's total trust value suddenly decreases in line with the current trust value. This behavior is consistent with the

V. EXPERIMENT AND ANALYSIS

This section conducts testing and analysis of the effectiveness and efficiency of the RTTAC model. The experimental environment is as follows:

Hardware Environment: CPU: Intel(R) Xeon(R) Gold 5218R CPU @ 2.10 GHz, RAM: 16 GiB, Operating System: Ubuntu 20.04.4 LTS; Software Environment: Docker 20.10.12, Hyperledger Fabric 2.2.0, Go version: 1.18.

A. Trust analysis

In the third section of this paper, it is explained how the trust level of users in the system is assessed. In the RTTAC model, the comparative importance of the three characteristics and the evidence within them is as follows: Security Characteristic $S >$ Reliability Characteristic $R >$ Performance Characteristic P , Performance Characteristic: $p1 > p2 > p3 > p4 > p5$, Reliability Characteristic: $r1 > r2$, Security Characteristic: $s1 > s2 > s3$.

Figure 4 illustrates the trend in current trust value and the total trust value as the proportion of negative behaviors by users increases. With the increase in the proportion of negative behaviors, both the current trust value and the total trust value of the users decrease. Figure 4 indicates that when the proportion of negative behaviors by users is relatively low, trust values decrease slowly. However, when users engage in high-risk negative behaviors, resulting in a higher proportion of negative behaviors, trust values decrease more rapidly, and the specific numerical values of both trust values become lower than before. These experimental data align with the trust value analysis results in Section 3.2 and are consistent with real-world observations.

expectations of the model's design and reflects real-world access situations.

B. Evaluation

1) Performance Analysis

We conducted experiments based on Hyperledger Fabric, simulating policy decisions and role transfers under different block configurations and transaction counts. Figure 5 displays the performance results of the RTTAC model under varying transaction counts. As transaction count increases, the throughput for policy decisions remains in the range of 200-245

TPS (Transactions Per Second). Under different process counts, we evaluated the average latency performance of smart contracts related to roles and policies, as shown in Figure 6. The average latency for read operations is lower than for write operations, and with an increase in the number of processes, both read and

write operation latencies show a slight increase. For role-related operations, the latency for read operations can be controlled within 60ms, while write operations remain within 730ms. Policy-related read operation latency also stays within 60ms, with a maximum of 680ms for write operations.

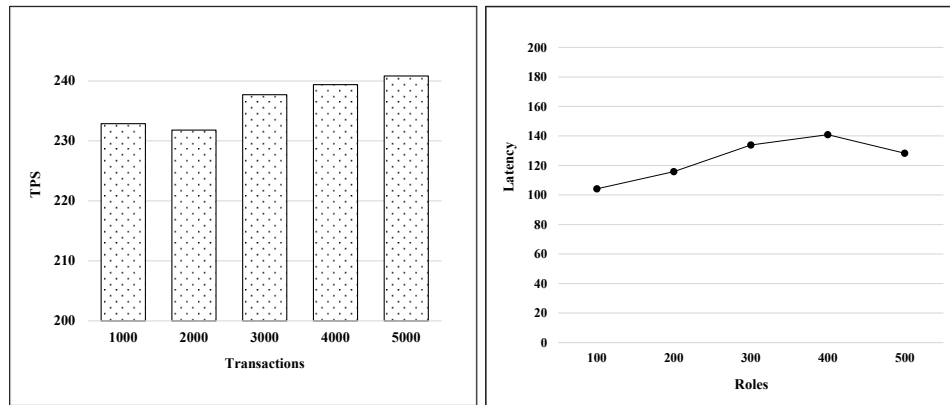


Figure 5. Performance analysis charts for access decisions and role transfers

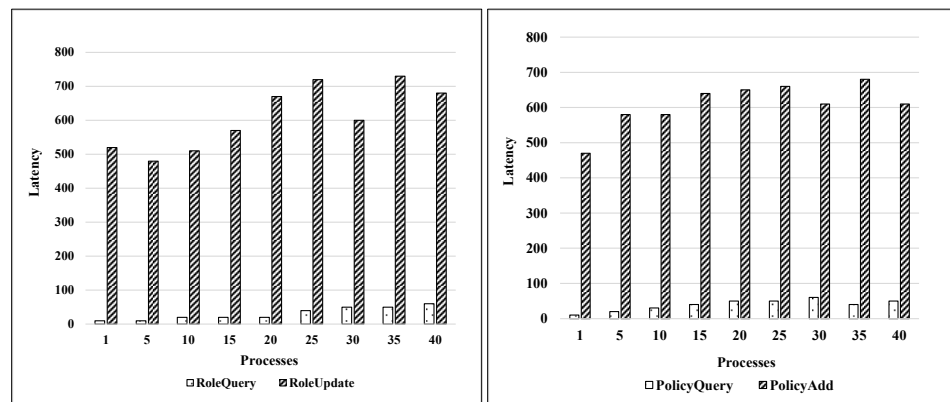


Figure 6. Average Latency of RMC and PMC under Different Processes

Regarding the role transfer process, the RTTAC model defines a one-to-many or one-to-one relationship between roles and data, while data has a one-to-one relationship with policies. Therefore, the number of roles may not necessarily match the number of policies. The experiments tested the average latency of role transfers within the range of 100 to 500 roles, as shown in Figure 5. The latency changes are relatively stable, and the overall average latency does not exceed 150ms. This indicates that the model can efficiently achieve role transfers in most scenarios.

VI. CONCLUSION

The RTTAC model proposed in this paper incorporates user trustworthiness into role attributes and implements a trusted role transfer mechanism, avoiding redundant roles and arbitrary assignments. In terms of access control, it combines the attribute-based access control model to achieve dynamic fine-grained access control. Experimental results show that the model can effectively limit unauthorized user access, ensuring data security, and enabling role transfer while maintaining system stability. This model has the potential to promote data-sharing applications on blockchain platforms.

ACKNOWLEDGMENT

The authors are supported by the Key Laboratory of Network Password Technology in Henan Province, China (LNCT2022-A20) in 2022; the Major Science and Technology Special Project of Henan Province, China (No.201300210100, No.201300210200); the Key Scientific Research Project of Higher Education Institutions in Henan Province, China (No.23ZX017); the Key Special Project for Science and Technology Collaborative Innovation in Zhengzhou City, Henan Province, China (No.21ZZXTCX07); and the Key Science and Technology Project of Henan Province, China (No.232102211082).

REFERENCES

- [1] Guo H, Yu X. A Survey on Blockchain Technology and its security[J]. Blockchain: research and applications, 2022, 3(2): 100067.
- [2] Monaco J V. Identifying bitcoin users by transaction behavior[C]. Biometric and surveillance technology for human and activity identification XII. SPIE, 2015, 9457: 25-39.
- [3] Sandhu R S, Coyne E J, Feinstein H L, et al. Role-based access control models[J]. Computer, 1996, 29(2): 38-47.

- [4] Al-Kahtani M A, Sandhu R. A model for attribute-based user-role assignment[C]//18th Annual Computer Security Applications Conference, 2002. Proceedings. IEEE, 2002: 353-362.
- [5] Jin X, Sandhu R, Krishnan R. RABAC: role-centric attribute-based access control[C]. Computer Network Security: 6th International Conference on Mathematical Methods, Models and Architectures for Computer Network Security, MMM-ACNS 2012, St. Petersburg, Russia, October 17-19, 2012. Proceedings 6. Springer Berlin Heidelberg, 2012: 84-96.
- [6] Rajpoot Q M, Jensen C D, Krishnan R. Integrating attributes into role-based access control[C]. Data and Applications Security and Privacy XXIX: 29th Annual IFIP WG 11.3 Working Conference, DBSec 2015, Fairfax, VA, USA, July 13-15, 2015, Proceedings 29. Springer International Publishing, 2015: 242-249.
- [7] Liang X, An N, Li D, et al. A Blockchain and ABAC Based Data Access Control Scheme in Smart Grid[C], 2022 International Conference on Blockchain Technology and Information Security (ICBCTIS). IEEE, 2022: 52-55.
- [8] JIANG J H, ZHANG X, DENG H J, et al. Multi-department data sharing access control scheme on blockchain[J]. Computer Integrated Manufacturing Systems, 2022, 28(10): 3202-3211.
- [9] ZHANG L J, LIU Z D, XIE G, et al. Secure data sharing model based on smart contract with integrated credit evaluation. Acta Automatica Sinica, 2021, 47(3): 594-608.
- [10] Wang Hai-Yong, Pan Qi-Qing, Guo Kai-Xuan. Access control model based on blockchain and user credit[J]. Journal of Computer Applications, 2020, 40(6): 1674-1679.
- [11] Benet J. Ipfs-content addressed, versioned, p2p file system[J]. arXiv preprint arXiv:1407.3561, 2014.